

How We Stopped the Calendly Booking Spam

What happened, and exactly what we changed so it cannot keep happening.

IN ONE SENTENCE

Someone found our public booking link and used it to flood the calendar with fake meetings. We killed the exposed link, stopped printing any booking link in the website's code, blocked the abuser, and cleaned up the mess. Real visitors are unaffected.

PART 1

In Plain Language

What was happening

Someone was repeatedly creating fake bookings on our Calendly "CRMConnect Discovery" meeting, roughly one every one to three minutes. Each fake booking set off a chain reaction: Calendly sent a notification email, added an entry to the calendar, and created a Zoom meeting. The inbox filled up fast, dozens of "New Event" emails with obvious junk names like "Ryy Gdjj Gejd" and "dscdscds cdscsdc."

The person hid behind two disposable email addresses (mernamoh16@gmail.com and saya689.f@gmail.com). None of the bookings were real. It was pure noise, designed to clog the calendar and the inbox.

Why it was possible

Two facts combined to make this easy for the attacker.

First, the booking page is not ours. Calendly hosts it, not us. Once someone has the direct link to a Calendly meeting, they can book straight on Calendly. Nothing on our own website sits between them and the calendar, so none of our website's protections can stop the booking.

Second, we were handing out the link in public. Our scheduling link was written directly into the website's page code (the underlying text a browser reads to build the page) on 23 English pages, plus roughly 380 auto-translated copies. Anyone can right-click a web page and choose "view page source" to read that code. The link was sitting right there in plain sight, ready to be copied and reused forever.

A SIMPLE ANALOGY

Think of the booking link as a key to a door that Calendly owns, not us. We had photocopied that key and taped a copy to the front window of every page on our website. Anyone walking by could read it, take a copy, and let themselves in whenever they wanted, no matter what locks we added to our own building. The attacker simply grabbed a copy of the key.

We had actually changed the link once before for this same reason (the old one ended in "-apac2"). That taught us an important lesson: just swapping the key does no good if we keep taping a fresh copy back onto the window. The real problem was that the key kept getting reprinted into the public page code.

What we did about it

We made five changes, working from "stop the bleeding" to "make sure it stays fixed."

- 1 We changed the lock, so the stolen key stopped working.**
We rotated the Calendly link to a new, hard-to-guess address. The old link the attacker had saved now returns a "page not found" error, so their saved copy died instantly.

- 2 We stopped printing the key in public.**
The website's page code no longer contains the booking link at all. It ships an empty placeholder instead. Only when a real visitor actually opens the scheduler does the page quietly ask our own server for the link, and our server hands it over only if the request genuinely came from one of our pages. An automated program that just reads the page code gets nothing.

- 3 We made future changes a one-step job.**
The link now lives in a single secure server setting. If a link is ever abused again, we swap it in one place. No rebuilding or redeploying the website.

- 4 We blocked the spammer in two places.**
At Calendly, we added both addresses to its built-in block list, which also automatically cancelled all of that person's upcoming fake meetings. And on our own website contact form, submissions from those addresses are now silently dropped.

- 5 We cleaned up the mess.**
We cancelled the remaining fake meetings and moved 59 spam notification emails to the Trash.

WHERE THINGS STAND NOW

The booking spam has stopped. The last fake booking came from sessions the attacker had already opened just before we changed the link, and nothing new has come through since. No booking link is exposed in any of the website's page code anymore. The link is revealed only at the moment a real visitor opens the scheduler. For genuine visitors, the "Schedule a Demo" experience is unchanged and works exactly as before.

Could it happen again?

Honestly, a very determined attacker could still recover the current link, but only by actually opening the scheduler in a real web browser, not by simply reading the page code. That is far more effort than before, and it defeats the cheap automated approach the spammer was using. If it ever does happen again, our fix is now a single one-line change instead of a site-wide edit. And if the problem keeps recurring, the next step available to us is to require email verification before the scheduler is shown at all.

PART 2

Technical Detail

Root cause

The scheduler was embedded as a hardcoded Calendly inline-widget: a `<div>` with a `data-url` attribute pointing at the `calendly.com` scheduling URL, plus Calendly's `widget.js` script, sitting in static HTML. This embed was present on 23 English product pages and roughly 380 auto-translated copies. Because the scheduling URL was literally in the served HTML, it was trivially harvestable via "view source," and the booking action then executed entirely on Calendly's infrastructure, outside any control we have on our own site. Rotating the URL alone had already proven insufficient (the prior "-apac2" link), because each rotation was immediately re-exposed in the markup.

The new architecture

The embed was replaced with a server endpoint plus a lazy client loader, so the URL is never present in the HTML and is only released to legitimate browser contexts.

Server component: `get-calendly-url` (Supabase edge function)

- Maps short event keys (for example `crmconnect`) to full Calendly URLs.
- The full URLs are stored as project **secrets** (environment variables such as `CALENDLY_URL_CRMCONNECT`), never in code and never in the page.
- Enforces a browser-context check: the request must carry an allowed `Origin` and/or `Referer` header from one of our own domains. Otherwise it returns `HTTP 403 Forbidden`.
- Applies a per-IP rate limit of 20 requests per hour.
- Logs every accept and reject as a structured line for later auditing.

Client component: `js/calendly-loader.js`

- Uses an `IntersectionObserver` to detect when a Calendly placeholder scrolls into view.
- On visibility, fetches the URL from `get-calendly-url`.
- Appends the page-specific query parameters (for example prefilled context and `hide_gdpr_banner`).

- Loads Calendly's `widget.js` once, then calls `Calendly.initInlineWidget` to render the scheduler.

Net effect: scrapers that only parse served HTML get an empty placeholder; only a rendered browser on an allowed origin, reaching the visible placeholder, ever receives the URL, and even then only within the rate limit.

Rotation as configuration

Rotating a link is now a configuration change, not a code change:

1. Rename the event slug inside Calendly. The old URL then returns `404` ; the new one returns `200` .
2. Update the corresponding secret (for example `CALENDLY_URL_CRMCONNECT`) to the new URL.

No site redeploy is required, because the value is read at runtime from the secret. The main event's new slug is `crmconnect-discovery-.....` (rotated; value not printed here) . In total, three exposed events were rotated to new hard-to-guess addresses: the main CRMConnect Discovery event and two further events found linked in blog posts (a second CRMConnect Discovery event and the CRMConnect Enterprise Discovery event). All old addresses now return `404` , and the new ones serve normally.

Additional hardening

Lead form blocklist

The `submit-lead` edge function now consults a `BLOCKED_EMAILS` secret (comma-separated). A blocked sender receives the same benign success response as our existing honeypot path, so they cannot tell they were blocked, and nothing is persisted or relayed downstream. A small dependency-free helper, `parseEmailBlocklist` , with unit tests, parses the list. Both throwaway addresses were added.

Calendly native control

Calendly's built-in invitee blocklist (Account Settings > Security > Blocked sources) was used to block both addresses. Blocking also auto-cancels the blocked party's future non-group meetings, which cleared the pending fake bookings.

Blog and cleanup

An audit of every page found three blog posts still carrying older Calendly links. Blog HTML is generated from a Supabase `blog_posts` table (the `body_md` Markdown column), so editing the generated HTML would be overwritten on the next rebuild. The fix was made in `body_md` , replacing those links with links to the relevant product pages, then the blog build was regenerated and deployed.

For inbox cleanup, 59 Gmail threads of spam notifications were moved to Trash via the Gmail API. They are recoverable for 30 days and were not hard-deleted.

Verification and residual risk

Verification: the old harvested URLs return `404` ; the new slugs return `200` ; no Calendly URL remains in any served HTML; the scheduler still renders correctly for legitimate visitors. The final fake booking traced to sessions the attacker opened just before rotation, and no new spam has arrived since.

RESIDUAL RISK

A determined attacker could still recover the current URL by opening the scheduler in a real browser (executing the client loader against an allowed origin), rather than scraping static HTML. This raises attacker effort substantially and defeats naive scraping. If abuse recurs, response is now a single one-line secret update. The next available escalation is to require email verification before the scheduler is revealed.

Reference

Item	Value
Commit	<code>0afa7696ee</code> (one commit, successful deploy)
Incident issue	GitHub issue #33
New endpoint	<code>get-calendly-url</code> (Supabase edge function)
Client loader	<code>js/calendly-loader.js</code>
Lead form function	<code>submit-lead</code> (edge function)
Key secrets (by name)	<code>CALENDLY_URL_CRMCONNECT</code> , <code>BLOCKED_EMAILS</code>
New main slug	<code>crmconnect-discovery-.....</code> (rotated; value not printed here)
Rate limit	20 requests per hour per IP
Links rotated	3 (main CRMConnect Discovery, plus 2 linked in blog posts)
Emails trashed	59 Gmail threads (recoverable 30 days)